and block from the wrapper. Note how we call the method by binding the method object to the current instance and then invoking that bound method.

The key to understanding this code, and most metaprogramming code, is to follow the basic principles we worked out at the start of this chapter—how self changes as methods are called and classes are defined and how methods are called by looking for them in the class of the receiver. If you get stuck, do what we do and draw little boxes and arrows. We find it useful to stick with the convention used in this chapter: class links go to the right, and superclass links go up. Given an object, a method call is then a question of finding the receiver object, going right once, and then following the superclass chain up as far as you need to go.

Top-Level Execution Environment

Finally, there's one small detail we have to cover to complete the metaprogramming environment. Many times in this book we've claimed that everything in Ruby is an object. However, we've used one thing time and time again that appears to contradict this—the top-level Ruby execution environment:

```
puts "Hello, World"
```

Not an object in sight. We may as well be writing some variant of Fortran or Basic. But dig deeper, and you'll come across objects and classes lurking in even the simplest code.

We know that the literal "Hello, World" generates a Ruby String, so that's one object. We also know that the bare method call to puts is effectively the same as self.puts. But what is self?

```
self.class # => Object
```

At the top level, we're executing code in the context of some predefined object. When we define methods, we're actually creating (private) instance methods for class Object. This is fairly subtle; as they are in class Object, these methods are available everywhere. And because we're in the context of Object, we can use all of Object's methods (including those mixed in from Kernel) in function form. This explains why we can call Kernel methods such as puts at the top level (and indeed throughout Ruby); it's because these methods are part of every object. Top-level instance variables also belong to this top-level object.

What's Next

Metaprogramming is one of Ruby's sharpest tools. Don't be afraid to use it to raise up the level at which you program. But, at the same time, use it only when necessary—overly metaprogrammed applications can become pretty obscure pretty quickly.

There's one more piece of the Ruby metaprogramming puzzle: reflection, which is how Ruby knows things about the runtime environment. Let's take a look.